EMBEDDED SYSTEM DESIGN EXPERIMENT

嵌入式系统设计实验

实验 06 基于RTOS的串口通信实验

杭州電子科技大学
HANGZHOU DIANZI UNIVERSITY

串行通信接口简介

串行接口简称串口，也称串行通信接口（通常指COM接口），是采用串行通信方式的扩展接口。

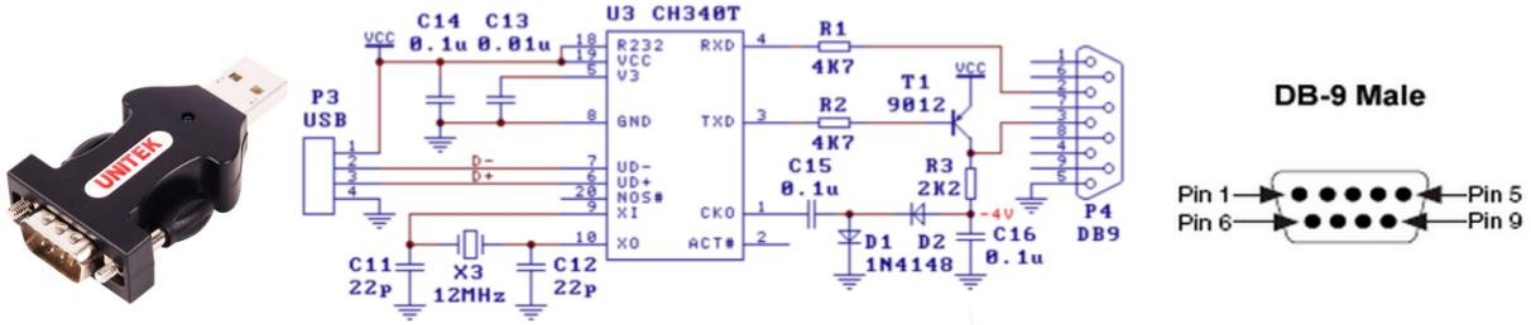
串口通信的两种最基本的方式：同步串行通信方式和异步串行通信方式。

1.同步串行是指SPI（Serial Peripheral interface）的缩写，顾名思义就是串行外围设备接口。SPI总线系统是一种同步串行外设接口，它可以使MCU与各种外围设备以串行方式进行通信以交换信息。

2.异步串行是指UART（Universal Asynchronous Receiver / Transmitter），通用异步接收/发送。UART包含TTL电平的串口和RS232电平的串口。 TTL电平是3.3V的，而RS232是负逻辑电平，它定义+5~+12V为低电平，而-12~-5V为高电平，通常PC机串口与单片机串口通信需要电平转换芯片如MAX232。

USB虚拟串口

如下图所示，现今的嵌入式设备和大多数开发板都采用外置USB转232器件，将单片机这边的UART串口转接到计算机USB口上，然后在计算机这边使用串口工具软件访问USB转接器件虚拟出来的USB串口即可实现计算机和单片机之间的串口通信。



更进一步的设计，则使用一个单片机实现程序下载、调试和USB虚拟串口的功能。课程配套学习板上的中下有一个STM32F103C8T6单片机，其作用就是实现一个DAP调试下载器，并且同时提供了一个USB虚拟串口功能，将STM32F4的串口1虚拟成了一个可被Windows识别的USB串口。有了这个串口，开发人员就可以通过串口调试软件和学习板进行通信了。

注意：USB虚拟串口一般在Windows 10以上系统中可以自动识别并安装驱动，然而Win7、Win8系统需要另外安装驱动。

串行通信的基本参数

串行端口的通信方式是将字节拆分成一个接一个的位再传输出去，接收方再将此一个一个的位组合成原来的字符，如此形成一个字节的完整传输，在数据传输时，应在通信端口的初始化时设置几个通信参数。

1）波特率，即传送数据的速度。波特率的意思就是在一秒中可以传输的数据位数，单位是bps。如果采用波特率4800bps进行传输，那么每秒可以传输600个byte。

2）数据位，当接收设备收到起始位后，紧接着就会收到数据位，数据位的个数可以是5、6、7或者8位。在字符数据传输的过程中，数据位从最低有效位开始传输。

3）起始位，在串口线上，没有数据传输时处于逻辑“1”状态，当发送设备要发送一个字符数据时，首先发出一个逻辑“0”信号，这个逻辑低电平就是起始位。

4）停止位，是一个字符数据的结束标志，它可以是1位、1.5位或者2位。

5）奇偶校验位，数据位发送完之后，就可以发送奇偶校验位。奇偶校验用于有限差错校验，通信双方在通信时约定一致的奇偶校验方式。

实验板虚拟串口介绍

实验板上的DAP下载器支持将STM32F4器件的串口1连接到左侧的USB Type-C接口并虚拟为一个USB串口。如下图所示，在Win10以上系统中可自动识别出该串口（Win7系统需要安装驱动）。

此次实验我们使用Windows下的串口调试助手软件和实验板进行串口通信。同时，实验板上的调试串口跳线必须接上。

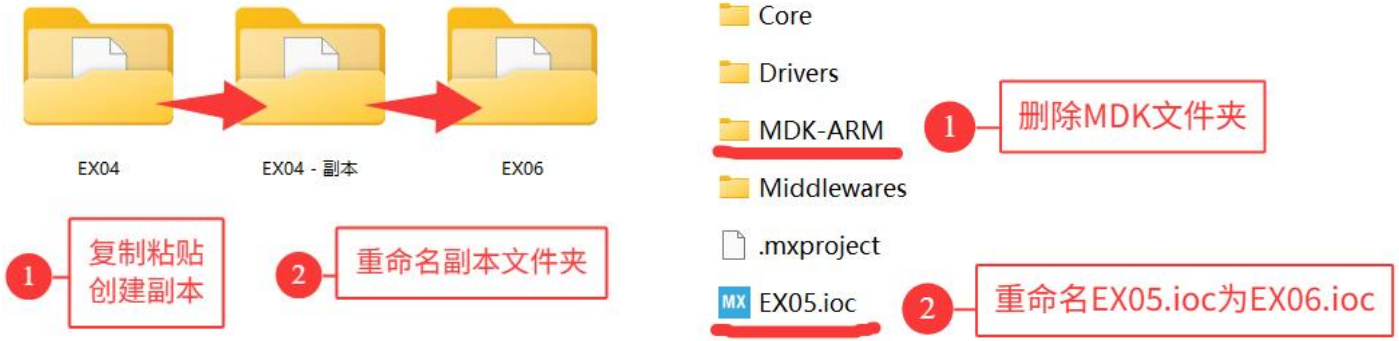
创建EX06示例工程并添加串口和设置RTOS

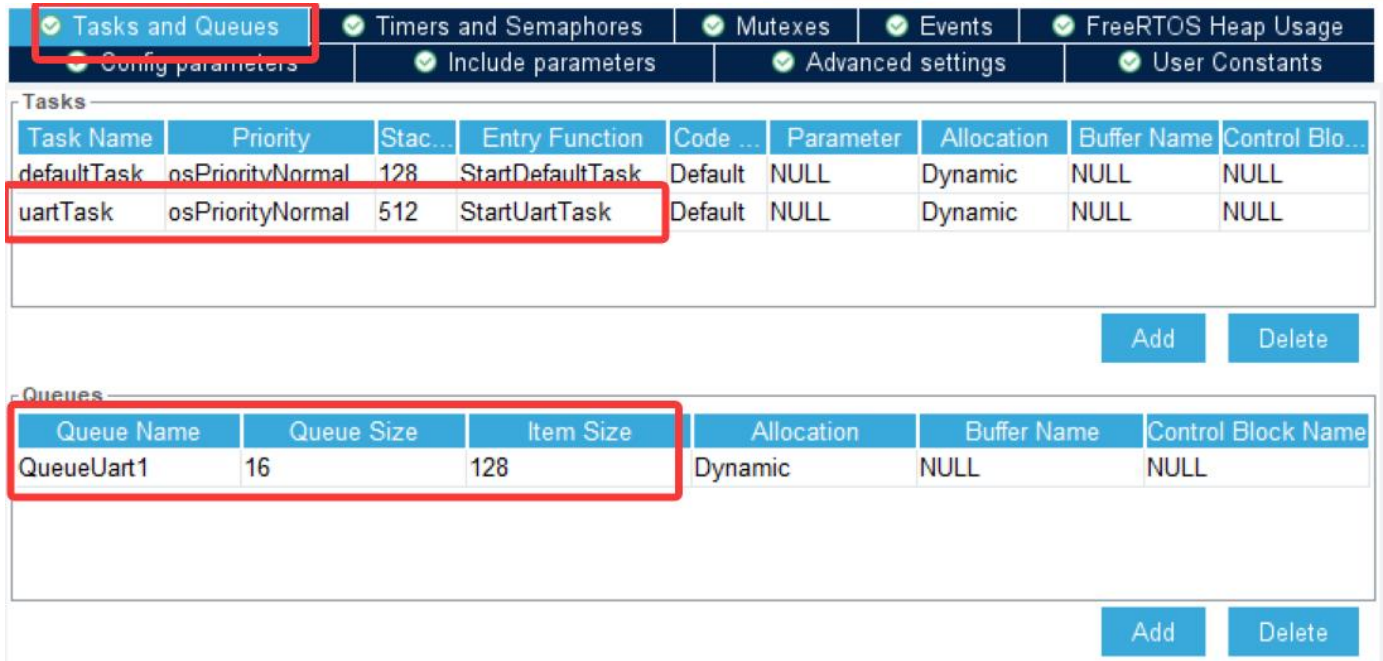
为减少每次新建工程的重复操作，可以按如下方法快速创建工程：

1. **复制粘贴**之前实验4的EX04工程文件夹，然后将副本**改名为EX06**。

2. **删除**EX06下的**MDK-ARM子目录**，并将原先的EX06.**ioc文件**，**改名**为EX006.ioc文件。

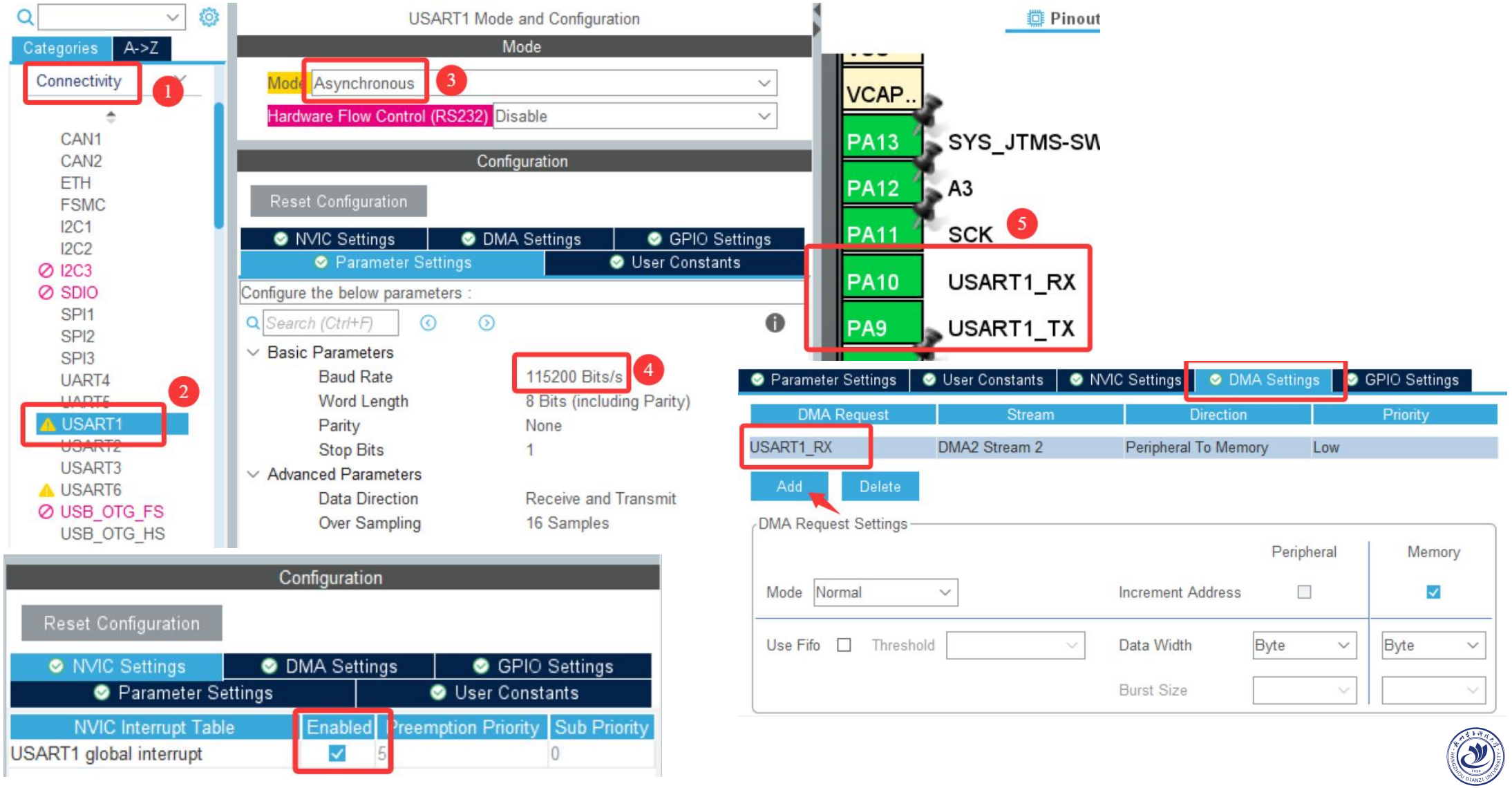
3. 双击EX06.ioc文件图标，启动STM32CubeMX软件打开工程，删除FreeRTOS中的数码管和按键扫描任务，参考右图，**设置串口任务和消息队列。**





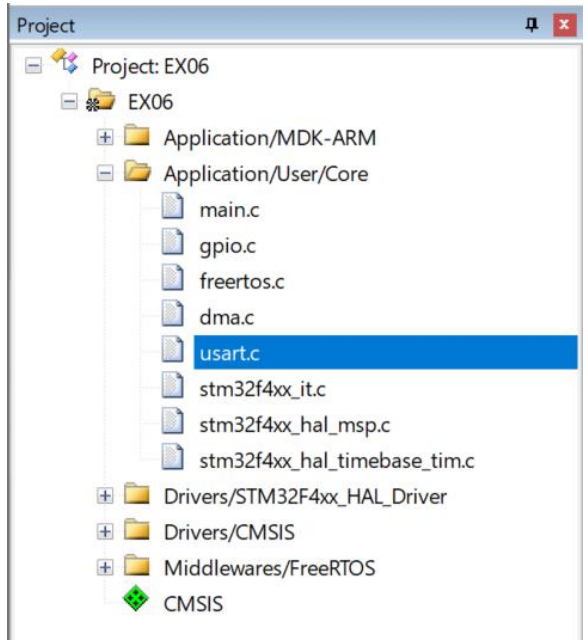
创建EX06示例工程并添加串口和设置RTOS

参考下图，**设置USART1**串口外设的工作模式和中断、DMA等工作参数，最后导出MDK工程。



添加串口打印输出

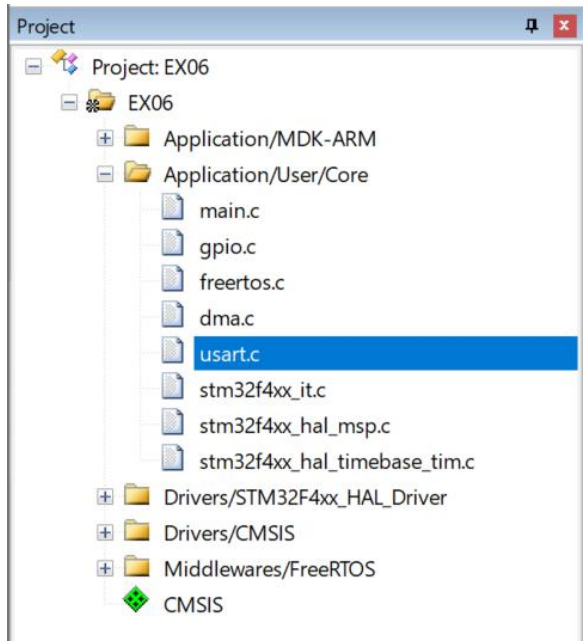
用MDK打开导出的EX06工程后，可以看到左侧工程文件列表多出了一个usart.c源文件，双击打开usart.c文件，向文件中添加fputc重载函数以支持printf打印到串口输出：




```
/* USER CODE BEGIN 0 */
#include <stdio.h>
#ifndef __GNUC__                // 如果不使用ARM CC V6编译器
#pragma import(__use_no_semihosting)  // 不使用半主机函数
struct __FILE{int handle;};  // 标准库需要的支持函数
void _sys_exit(int x) {x = x;}      // 避免使用半主机模式
#endif
FILE __stdout;                      // ARM CC V6版本需要添加支持
int fputc(int ch, FILE *f) {
        HAL_UART_Transmit(&huart1 , (uint8_t *)&ch, 1, 0xFFFF);
        return(ch);
}
/* USER CODE END 0 */
```

添加串口打印输出（arm-none-eabi-gcc编译器）

如果使用Clion或VSCode进行STM32开发，则需双击打开usart.c文件，向文件中添加__io_putchar重载函数以支持printf打印到串口输出：




```
/* USER CODE BEGIN 0 */
#ifdef __GNUC__
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif

PUTCHAR_PROTOTYPE
{
  HAL_UART_Transmit(&huart1, (uint8_t *) &ch, 1, HAL_MAX_DELAY);
  return ch;
}
/* USER CODE END 0 */
```

测试串口输出

修改freertos.c文件，先添加包含stdio.h头文件，清除默认任务中原有代码，然后添加代码定时打印当前OS运行时间。添加代码如下所示：

```
/* Private includes ----------*/
/* USER CODE BEGIN Includes */
#include "stdio.h"
/* USER CODE END Includes */
...

void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  uint32_t ot = 500;
  for(;;) {
    uint32_t ct = osKernelGetTickCount();
    if (ct >= ot) {
      ot = ct + 500;
      printf("Run time is %.3fs\n", ct / 1000.0f);
    }
    osDelay(1);
  }
  /* USER CODE END StartDefaultTask */
}
```

编译成功后下载运行程序，在PC端使用串口调试工具软件，打开虚拟串口并设置相同波特率，接收数据如下图：

字符串格式化浮点数支持（arm-none-eabi-gcc编译器）

如果使用Clion或VSCode进行STM32开发，使用printf打印浮点数时，需要修改CMakeList.txt文件或.config配置文件。添加编译选项如下所示：

Clion中，修改CMakeList.txt文件，在文件末尾添加编译选项：

```
# 浮点数字符串格式化支持
target_link_options(${CMAKE_PROJECT_NAME} PRIVATE
                    -u _printf_float
)
```

在**vscode**+“stm32 for vscode”插件编译时，修改STM32-for-VSCode.config文件，找到linkerFlags行，在链接选项末尾添加,-u_printf_float

比如：

```
linkerFlags:
  - -Wl,--print-memory-usage,-u_printf_float
```

串口通信API

STM32 HAL库提供的串口操作函数如下表所示。发送数据时通常使用HAL_UART_Transmit函数，而接收数据根据接收方法不同，在不同的地方采用不同的接收函数。

函数名称	说明
HAL_UART_Transmit	串口发送数据
HAL_UART_Receive	串口接收数据
HAL_UART_Transmit_IT	串口带中断方式发送数据
HAL_UART_Receive_IT	串口带中断方式接收数据
HAL_UART_Transmit_DMA	串口DMA方式发送数据
HAL_UART_Receive_DMA	串口DMA方式接收数据
HAL_UARTEx_ReceiveToIdle	串口接收数据，空闲或超时退出
HAL_UARTEx_ReceiveToIdle_IT	串口带中断方式接收数据，空闲时进入中断事件回调函数
HAL_UARTEx_ReceiveToIdle_DMA	串口DMA方式接收数据，空闲时进入中断事件回调函数
HAL_UART_IRQHandler	串口中断服务函数
HAL_UART_TxCpltCallback	串口发送中断回调函数
HAL_UART_RxCpltCallback	串口接收中断回调函数
HAL_UARTEx_RxEventCallback	串口接收空闲中断事件回调函数

STM32的串口通信方式

从上一页的表中可以看出STM32的串口通信方式大致有轮询、中断和DMA三种方式，而且DMA接收方式也需要开启串口中断。

轮询方式适合简单的串口通信，但是在复杂多任务系统不适合使用，数据接收会受到高优先级任务影响，或者影响其它低优先级任务。

中断和**DMA**接收方式，都需要调用接收中断回调函数HAL_UART_RxCpltCallback或者HAL_UARTEx_RxEventCallback，用户需要重新编写该函数，以便在接收数据到来时及时处理。

STM32定义串口总线上在一个字节的时间内没有再接收到数据时，认为是串口空闲；而空闲中断是检测到有数据被接收后，总线上在一个字节的时间内没有再接收到数据的时候发生的。比如在通信速率为115200bps时，一个字节的时间可以大致计算为1000 * 10 / 115200 = 0.087ms，取整约等于100微秒。

轮询接收方式串口通信

在一个循环中调用**HAL_UART_Receive**函数，以阻塞方式读取串口数据，当串口接收到数据时返回HAL_OK。示例如下：

```
/* USER CODE BEGIN Includes */
#include "usart.h"
#include "stdio.h"
/* USER CODE END Includes */
...

void StartDefaultTask(void *argument) {
        /* USER CODE BEGIN StartDefaultTask */
        uint8_t ch;          // 临时变量
        for(;;) {
                if (HAL_UART_Receive(&huart1, &ch, 1, 10) == HAL_OK)        // 读取1字节数据
                        printf("%c", ch);    // 打印接收字符
                // osDelay(1);                  // 不能加延时，加了更容易丢字符
        }
        /* USER CODE END StartTaskUart */
}
```

如上图所示，测试字符串加长时会发现接收数据不全，因为printf打印数据耗时较大会导致漏接数据

轮询接收方式串口通信

相比每次单字节的接收处理方式，采用**HAL_UARTEx_ReceiveToIdle**函数进行轮询接收处理方式会更好一点，修改代码如下。函数从串口1中开始接收数据后，自动将接收数据连续存储到缓冲数组内，当接收完指定字节数或者串口空闲时函数返回，返回时更新参数len变量的数值（实际接收数据长度），最后一个参数表示接收超时等待20毫秒。

```
/* USER CODE END Header_StartTaskUart */
void StartDefaultTask(void *argument) {
        /* USER CODE BEGIN StartDefaultTask*/
        uint8_t buf[128];  // 定义接收缓冲数组
        uint16_t len;      // 定义临时变量，表示接收数据长度
        for(;;)  {
                if (HAL_UARTEx_ReceiveToIdle(&huart1, buf, 128, &len, 20) == HAL_OK){
                        buf[len] = '\0';              // 接收数据末尾加字符串结束符
                        printf("%s", buf);            // 打印接收字符串
                }
                // osDelay(1);  // 延时不能加，容易丢数据
        }
        /* USER CODE END StartTaskUart */
}
```

中断接收方式串口通信——串口接收中断示例

中断方式需要用到接收缓冲，可以先在freertos.c文件中定义全局变量如下：

```
/* USER CODE BEGIN Variables */
uint8_t ch;        // 接收缓冲变量
/* USER CODE END Variables */
```

然后在任务循环开始前启动串口中断接收方式，设置每接收到1个字节进入中断，调用中断回调函数。在任务循环中定义临时数组读取消息并将其打印出来：

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  HAL_UART_Receive_IT(&huart1, &ch, 1);          // 开启中断接收方式
  for(;;)  {            char dat[128];           // 定义消息队列读取缓冲
            if (osMessageQueueGet(QueueUart1Handle, dat, NULL, 10) == osOK)
                      printf("%s", dat);          // 打印获取的消息
            osDelay(1);
  }
  /* USER CODE END StartDefaultTask */
}
```

中断接收方式串口通信——串口接收中断示例

串口接收中断回调函数名为HAL_UART_RxCpltCallback，该函数在串口接收中断发生时被串口中断函数所调用，它的函数原型在stm32f4xx_hal_uart.h定义如下：

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
```

现在freertos.c文件末尾重写该函数，并添加串口接收数据的消息发送功能：

```
/* USER CODE BEGIN Application */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) { // 注意函数名称不能错
        if (huart == &huart1)   {
                uint8_t buf[128] = {0};   // 定义消息发送缓冲
                buf[0] = ch;              // 写入接收的串口数据
                osMessageQueuePut(QueueUart1Handle, buf, NULL, 0); // 发送消息
                __HAL_UNLOCK(huart); // 解锁串口状态
                HAL_UART_Receive_IT(huart, &ch, 1);   // 重新开始中断接收
        }
}
/* USER CODE END Application */
```

中断接收方式串口通信——串口接收空闲中断示例

编译工程，程序下载测试结果表明，测试字符串长度超过16字节时有比较明显丢失。原因在于消息队列深度只有16，而每次接收到1字节数据时就向队列发送了一次消息，这样在接收到一长串字符数据时，就会因消息队列满溢而导致数据丢失。

对于这种情况，使用**HAL_UARTEx_ReceiveToIdle_IT**函数更为适合，该函数仅指定接收数据缓冲大小，当开始接收数据并超时空闲后才进入串口中断并调用接收中断空闲回调函数**HAL_UARTEx_RxEventCallback**。

修改freertos.c文件，在文件开头添加接收缓冲数组定义：

```
/* USER CODE BEGIN Variables */
uint8_t rx_buf[128];  // 接收缓冲数组
/* USER CODE END Variables */
```

在MX_FREERTOS_Init函数中，确认消息队列的消息大小为128：

```
/* creation of myQueue01 */
QueueUart1Handle = osMessageQueueNew (16, 128, &QueueUart1_attributes);
```

中断接收方式串口通信——串口接收空闲中断示例

然后修改任务函数，在循环开始前开启空闲超时接收中断方式：

```
void StartDefaultTask(void *argument) {
    /* USER CODE BEGIN StartDefaultTask */
    HAL_UARTEx_ReceiveToIdle_IT(&huart1, rx_buf, 127); // 开启空闲超时接收中断
    for(;;)  {
            char dat[128];
            if (osMessageQueueGet(QueueUart1Handle, dat, NULL, 10) == osOK)
                    printf("%s", dat);
            osDelay(1);
    }
    /* USER CODE END StartDefaultTask */
}
```

中断接收方式串口通信——串口接收空闲中断示例

最后在freertos.c末尾重写空闲中断回调函数**HAL_UARTEx_RxEventCallback**（函数名称注意不能拼写错误），并添加串口接收数据的消息发送功能：

```
/* USER CODE BEGIN Application */
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size) {
        if (huart == &huart1) {
                rx_buf[Size] = '\0';        // 接收数据末尾添加字符串结束符
                osMessageQueuePut(QueueUart1Handle, rx_buf, NULL, 0); // 发送消息
                __HAL_UNLOCK(huart); // 解锁串口状态
                HAL_UARTEx_ReceiveToIdle_IT(&huart1, rx_buf, 127);    // 再次开启接收
        }
}
/* USER CODE END Application */
```

编译工程后，程序下载测试结果良好。要注意的是，以上示例代码适用于字符串收发场景，如果要收发任意字节数据，程序还需进行优化调整。

中断接收方式串口通信——DMA

DMA全称Direct Memory Access，即直接存储器访问的简称。使用DMA能够提高CPU的利用率，省去了CPU搬运数据的时间，对于高效能嵌入式系统算法和网络应用非常重要。

使用STM32的串口DMA功能，除了之前在CubeMX中设置串口DMA请求，还需确认导出的MDK工程代码是否正确。打开main.c文件，确认main()函数中DMA初始化动作先于串口初始化动作执行，如果不是，需要手动调整一下，两条初始化语句顺序如下所示：

```
/* Initialize all configured peripherals */
MX_GPIO_Init();                 // GPIO端口初始化
MX_DMA_Init();                  // DMA初始化
MX_USART1_UART_Init();          // 串口初始化
/* USER CODE BEGIN 2 */
```

中断接收方式串口通信——串口接收的DMA空闲中断示例

STM32的串口DMA接收，可以使用HAL_UART_Receive_DMA函数开启DMA接收，或使用**HAL_UARTEx_ReceiveToIdle_DMA**函数开启DMA接收并同时开启空闲中断。接收不定长度的串口数据时，推荐使用后一个函数进行DMA接收。

HAL_UARTEx_ReceiveToIdle_DMA函数在开启串口DMA接收的同时还开启了串口空闲中断，当DMA接收指定数据完成或者判断串口空闲时将调用**HAL_UARTEx_RxEventCallback**回调函数。在freertos.c文件中，接收缓冲数组不变，修改任务函数代码如下所示：

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  HAL_UARTEx_ReceiveToIdle_DMA(&huart1, rx_buf, 126);          // 开启DMA空闲接收中断
  for(;;)  {
            ...          // 同空闲中断示例，保持不变
  }
  /* USER CODE END StartDefaultTask */
}
```

中断接收方式串口通信——串口接收的DMA空闲中断示例

最后在freertos.c末尾重写空闲中断回调函数**HAL_UARTEx_RxEventCallback**（函数名称注意不能拼写错误），并添加串口接收数据的消息发送功能：

```
/* USER CODE BEGIN Application */
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart, uint16_t Size) {
        if (huart == &huart1) {
                HAL_UART_DMAStop(huart);     // 暂停DMA接收数据
                rx_buf[Size] = '\0';         // 接收数据末尾添加字符串结束符
                osMessageQueuePut(QueueUart1Handle, rx_buf, NULL, 0);// 发送消息
                __HAL_UNLOCK(huart);         // 串口解锁
                HAL_UARTEx_ReceiveToIdle_DMA(&huart1, rx_buf, 126);    // 重新开始接收
        }
}
/* USER CODE END Application */
```

编译工程后，程序下载测试结果良好。相对而言，使用DMA空闲中断接收串口数据效果良好，配合应用消息队列能保证数据接收完整，处理及时。

流水灯串口通信应用示例

接下来在**串口空闲中断示例**的基础上，演示一个简单的流水灯串口通信应用。

修改StartDefaultTask任务函数代码如下：

```
void StartDefaultTask(void *argument) {
  /* USER CODE BEGIN StartDefaultTask */
  uint8_t sta, dir, brun, speed = 5;
  sta = dir = brun = 1;        // LED灯初始状态
  for(;;)  {
    osDelay((10 - speed) * 100);  // 流水灯间隔时间
    if (brun) {                   // 如果流水灯运行
      if (sta == 0x01 || sta == 0x80)
        dir = !dir;
      sta = dir ? (sta >> 1) : (sta << 1);
      // 打印流水灯状态
      for (int i = 0; i < 8; ++i)
        printf("%s", (sta & (0x01 << i)) ? "●" : "○");
          printf("\n");
    }
    SetLeds(sta);  // 调用亮灯函数，刷新LED灯
    // 等待任务通知，并设置10毫秒超时
    uint32_t flag =
            osThreadFlagsWait(0xFF, osFlagsWaitAny, 10);
    switch (flag) {
      case 0x10:  brun = 1;  break;     // 启动流水灯
      case 0x20:  brun = 0;  break;     // 暂停流水灯
      default:
                if (flag > 0 && flag < 10) // 流水灯速度
                  speed = flag;
                break;
    }
  }
  /* USER CODE END StartDefaultTask */
}
```

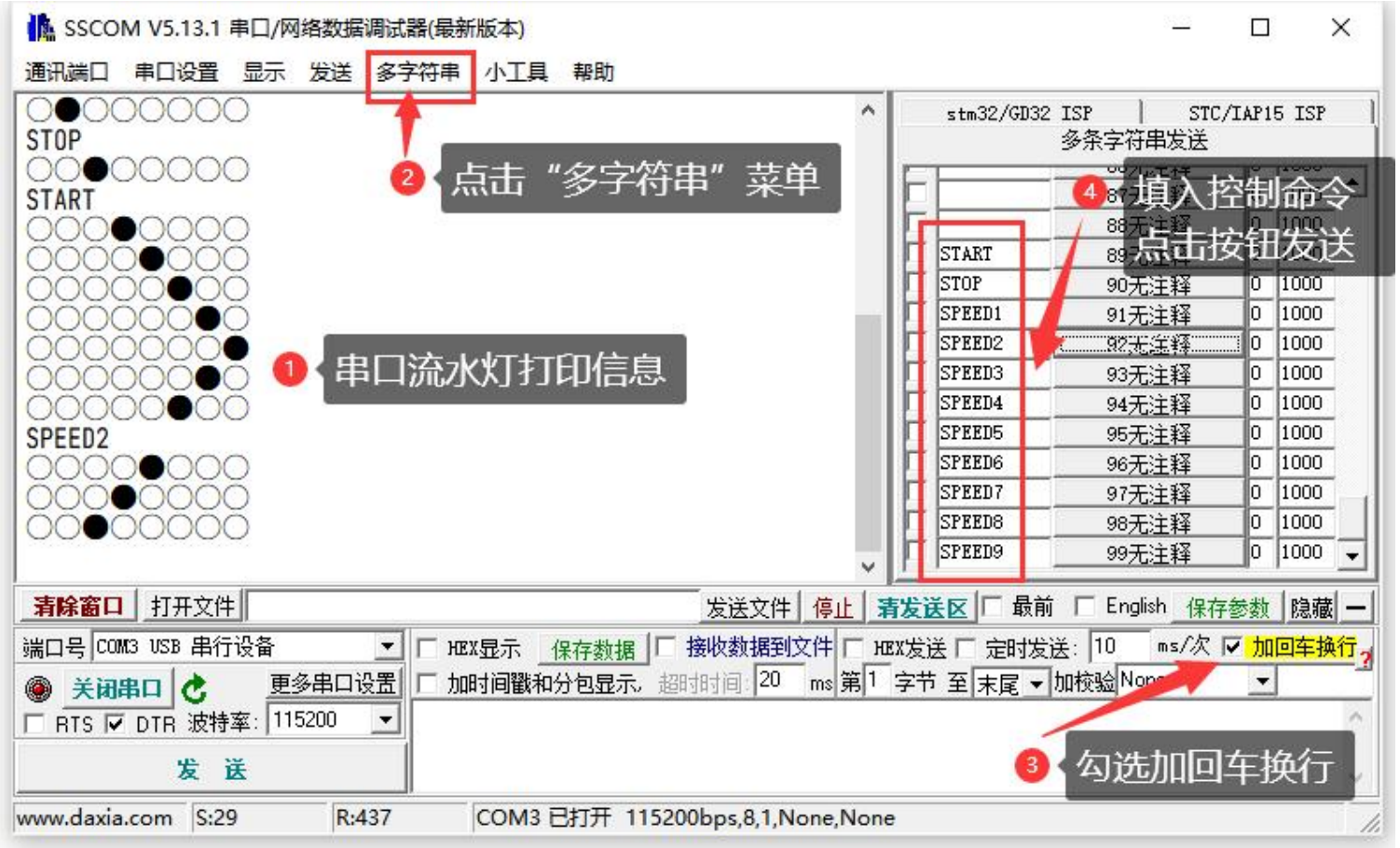
流水灯串口通信应用示例

编译工程后，下载程序到学习板上测试运行，可以看到流水灯已经自动运行起来了。现在修改**StartTaskUart**串口任务函数，添加接收字符串处理代码：

```
void StartUartTask(void *argument) {
  /* USER CODE BEGIN StartUartTask */
  HAL_UARTEx_ReceiveToIdle_IT(&huart1, rx_buf, 127); // 开启空闲中断接收
  for(;;) {       char dat[128];  // 定义消息队列读取缓冲
    if (osMessageQueueGet(QueueUart1Handle, dat, NULL, 10) == osOK)  {
      printf("%s", dat);  // 打印获取的消息
      // 对应不同的串口命令，向默认任务发送不同的任务通知
      if (strstr(dat, "START") == dat)
        osThreadFlagsSet(defaultTaskHandle, 0x10);
      else if (strstr(dat, "STOP") == dat)
        osThreadFlagsSet(defaultTaskHandle, 0x20);
      else if (strstr(dat, "SPEED") == dat) {
        uint8_t sp = dat[5] - '0';    // 提取速度数值
        if (sp > 0 && sp < 10)      // 将数值1-9发送通知给默认任务
          osThreadFlagsSet(defaultTaskHandle, sp);
      }
    }
    osDelay(1);
  }
  /* USER CODE END StartUartTask */
}
```

流水灯串口通信应用示例

重新编译程序，成功后下载到学习板上测试运行，现在串口调试助手中已经可以看到程序打印出来的流水灯状态了，并且向学习板发送对应的控制命令也能够得到及时响应，演示结果如下：



实验内容

1. 示例工程实践练习

参照本次实验讲义内容，创建并复现EX06的流水灯示例工程，最后在学习板上测试通过。

2. 数码管任务编程练习

添加一个数码管显示任务StartSegTask，使用任务通知或消息队列的通信方法实现串口通知数码管显示功能。

在串口调试助手中发送“SET:xxxx”(xxxx指0000 ~ 9999这样的4位10进制数)这样的设置指令，能控制数码管显示指定的数值。要求在串口任务中将接收提取的10进制数值通过任务通知或消息队列的方法传递给数码管任务进行显示。

思考题，如果xxxx是16进制数据，该如何接收提取并显示到数码管上？

3. 到学习通上的对应课程作业中，完成提交本次实验的**课堂作业**。

下一次实验07

实验07_GUI应用实验

请提前做好预习